

1

Microcomputer Systems

Overview

This chapter provides an introduction to the architecture of microcomputers in general and to the IBM PC in particular. You will learn about the main hardware components: the central processor, memory, and the peripherals, and their relation to the software, or programs. We'll see exactly what the computer does when it executes an instruction, and discuss the main advantages (and disadvantages) of assembly language programming. If you are an experienced microcomputer user, you are already familiar with most of the ideas discussed here; if you are a novice, this chapter introduces many of the important terms used in the rest of the book.

1.1 The Components of a Microcomputer System

Figure 1.1 shows a typical microcomputer system, consisting of a system unit, a keyboard, a display screen, and disk drives. The system unit is often referred to as "the computer," because it houses the circuit boards of the computer. The keyboard, display screen, and disk drives are called **I/O devices** because they perform input/output operations for the computer. They are also called **peripheral devices** or **peripherals**.

Integrated-circuit (IC) chips are used in the construction of computer circuits. Each IC chip may contain hundreds or even thousands of transistors. These IC circuits are known as **digital circuits** because they operate on discrete voltage signal levels, typically, a high voltage and a low voltage. We use the symbols 0 and 1 to represent the low- and high-voltage signals, respectively. These symbols are called **binary digits**, or **bits**. All information processed by the computer is represented by strings of 0's and 1's; that is, by bit strings.

Figure 1.1 A Microcomputer System



Functionally, the computer circuits consist of three parts: the **central processing unit (CPU)**, the memory circuits, and the I/O circuits. In a microcomputer, the CPU is a single-chip processor called a **microprocessor**. The CPU is the brain of the computer, and it controls all operations. It uses the memory circuits to store information, and the I/O circuits to communicate with I/O devices.

The System Board

Inside the system unit is a main circuit board called the **system board**, which contains the microprocessor and memory circuits. The system board is also called a **motherboard** because it contains **expansion slots**, which are connectors for additional circuit boards called **add-in boards** or **add-in cards**. I/O circuits are usually located on add-in cards. Figure 1.2 shows the picture of a motherboard.

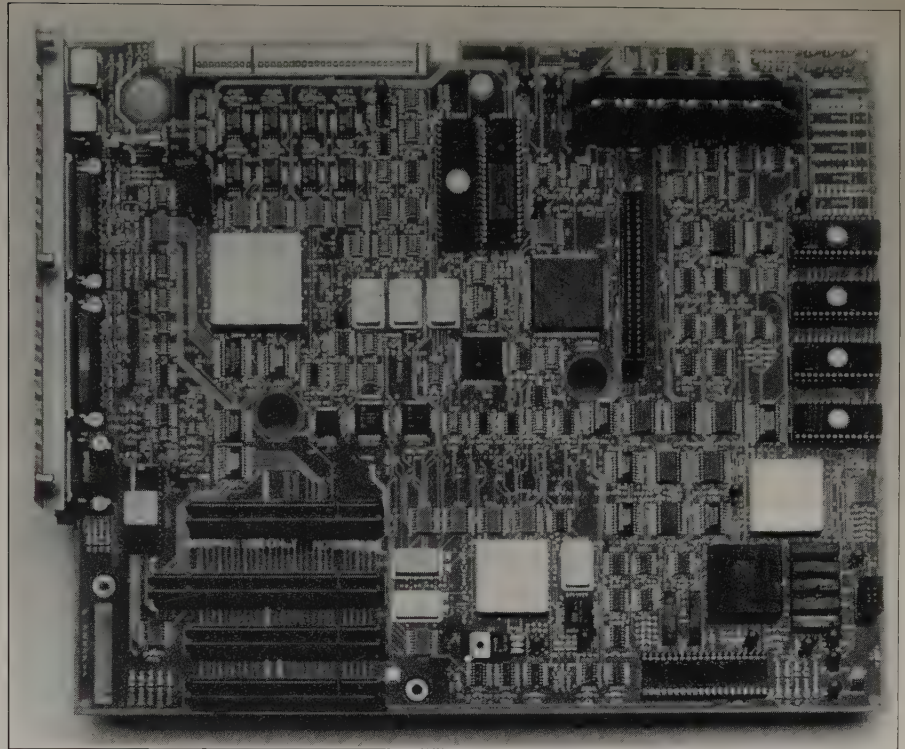
1.1.1

Memory

Bytes and Words

Information processed by the computer is stored in its memory. A memory circuit element can store one bit of data. However, the memory circuits are usually organized into groups that can store eight bits of data, and a string of eight bits is called a **byte**. Each **memory byte circuit**—or **memory byte**, for short—is identified by a number that is called its **address**, like the street address of a house. The first memory byte has address

Figure 1.2 A Motherboard



0. The data stored in a memory byte are called its **contents**. When the contents of a memory byte are treated as a single number, we often use the term *value* to denote them.

It is important to understand the difference between address and contents. The address of a memory byte is fixed and is different from the address of any other memory byte in the computer. Yet the contents of a memory byte are not unique and are subject to change, because they denote the data *currently* being stored. Figure 1.3 shows the organization of memory bytes; the contents are arbitrary.

Another distinction between address and contents is that while the contents of a memory byte are always eight bits, the number of bits in an

Figure 1.3 Memory Represented as Bytes

Address	Contents							
.	.	.	.	.	.	.	.	.
7	0	0	1	0	1	1	0	1
6	1	1	0	0	1	1	1	0
5	0	0	0	0	1	1	0	1
4	1	1	1	0	1	1	0	1
3	0	0	0	0	0	0	0	0
2	1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1	0
0	0	1	1	0	0	0	0	1

address depends on the processor. For example, the Intel 8086 microprocessor assigns a 20-bit address, and the Intel 80286 microprocessor uses a 24-bit address. The number of bits used in the address determines the number of bytes that can be accessed by the processor.

Example 1.1 Suppose a processor uses 20 bits for an address. How many memory bytes can be accessed?

Solution: A bit can have two possible values, so in a 20-bit address there can be $2^{20} = 1,048,576$ different values, with each value being the potential address of a memory byte. In computer terminology, the number 2^{20} is called **1 mega**. Thus, a 20-bit address can be used to address **1 megabyte** or **1 MB**.

In a typical microcomputer, two bytes form a **word**. To accommodate word data, the IBM PC allows any pair of successive memory bytes to be treated as a single unit, called a **memory word**. The lower address of the two memory bytes is used as the address of the memory word. Thus the memory word with the address 2 is made up of the memory bytes with the addresses 2 and 3. The microprocessor can always tell, by other information contained in each instruction, whether an address refers to a byte or a word.

In this book, we use the term **memory location** to denote either a memory byte or a memory word.

Bit Position

Figure 1.4 shows the bit positions in a microcomputer word and a byte. The positions are numbered from right to left, starting with 0. In a word, the bits 0 to 7 form the *low byte* and the bits 8 to 15 form the *high byte*. For a word stored in memory, its low byte comes from the memory byte with the lower address and its high byte is from the memory byte with the higher address.

Memory Operations

The processor can perform two operations on memory: *read* (fetch) the contents of a location and *write* (store) data at a location. In a read operation, the processor only gets a copy of the data; the original contents

Figure 1.4 Bit Positions in a Byte and a Word

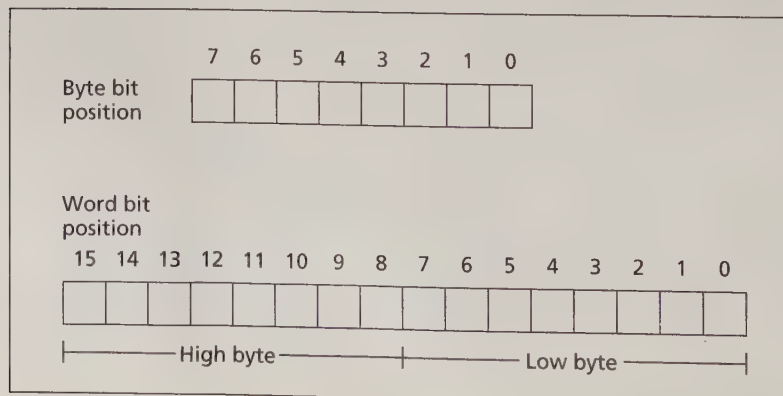
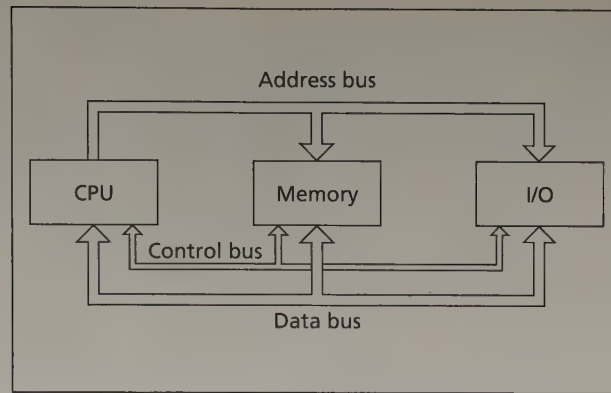


Figure 1.5 Bus Connections of a Microcomputer



of the location are unchanged. In a write operation, the data written become the new contents of the location; the original contents are thus lost.

RAM and ROM

There are two kinds of memory circuits: **random access memory (RAM)** and **read-only memory (ROM)**. The difference is that RAM locations can be read and written, while, as the name implies, ROM locations can only be read. This is because the contents of ROM memory, once initialized, cannot be changed.

Program instructions and data are normally loaded into RAM memory. However, the contents of RAM memory are lost when the machine is turned off, so anything valuable in RAM must be saved on a disk or printed out beforehand. ROM circuits retain their values even when the power is off. Consequently, ROM is used by computer manufacturers to store system programs. These ROM-based programs are known as **firmware**. They are responsible for loading start-up programs from disk as well as for self-testing the computer when it is turned on.

Buses

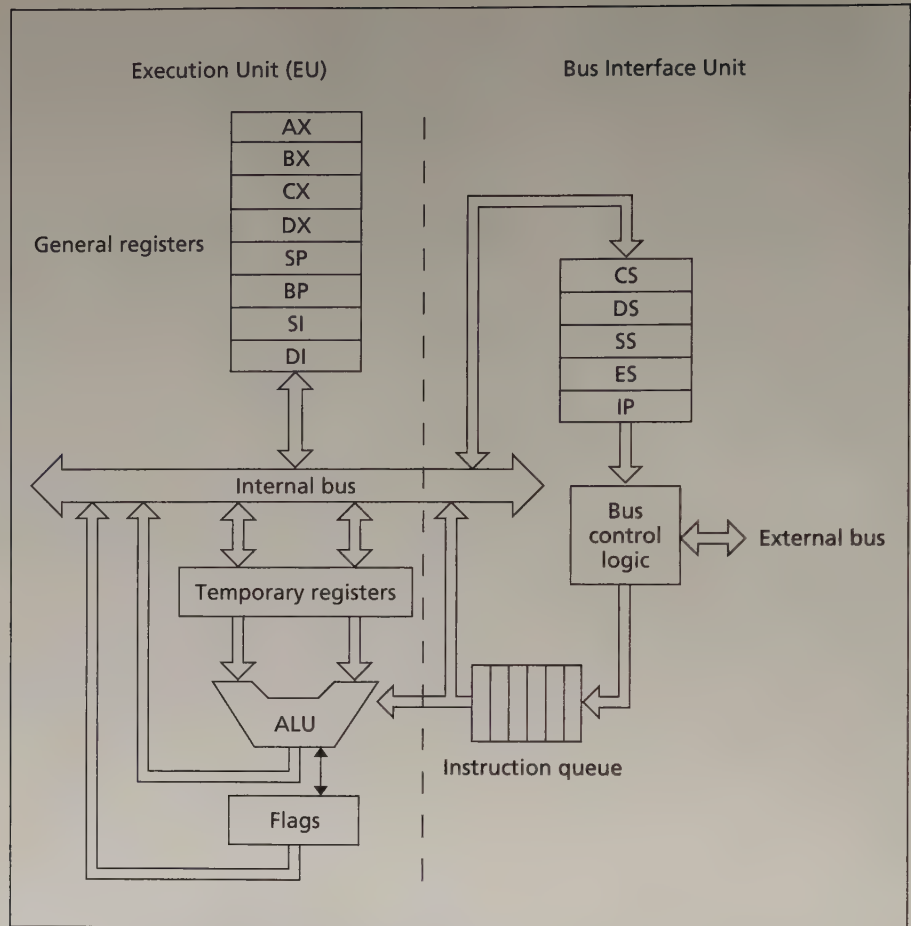
A processor communicates with memory and I/O circuits by using signals that travel along a set of wires or connections called **buses** that connect the different components. There are three kinds of signals: address, data, and control. And there are three buses: **address bus**, **data bus**, and **control bus**. For example, to read the contents of a memory location, the CPU places the address of the memory location on the address bus, and it receives the data, sent by the memory circuits, on the data bus. A control signal is required to inform the memory to perform a read operation. The CPU sends the control signal on the control bus. Figure 1.5 is a diagram of the bus connections for a microcomputer.

1.1.2

The CPU

As stated, the CPU is the brain of the computer. It controls the computer by executing programs stored in memory. A program might be a system program or an application program written by a user. In any case, each instruction that the CPU executes is a bit string (for the Intel 8086, instructions are from one to six bytes long). This language of 0's and 1's is called **machine language**.

**Figure 1.6 Intel 8086
Microprocessor Organization**



The instructions performed by a CPU are called its **instruction set**, and the instruction set for each CPU is unique. To keep the cost of computers down, machine language instructions are designed to be simple; for example, adding two numbers or moving a number from one location to another. The amazing thing about computers is that the incredibly complex tasks they perform are, in the end, just a sequence of very basic operations.

In the following, we will use the Intel 8086 microprocessor as an example of a CPU. Figure 1.6 shows its organization. There are two main components: the **execution unit** and the **bus interface unit**.

Execution Unit (EU)

As the name implies, the purpose of the execution unit (EU) is to execute instructions. It contains a circuit called the **arithmetic and logic unit (ALU)**. The ALU can perform arithmetic (+, -, ×, /) and logic (AND, OR, NOT) operations. The data for the operations are stored in circuits called **registers**. A register is like a memory location except that we normally refer to it by a name rather than a number. The EU has eight registers for storing data; their names are AX, BX, CX, DX, SI, DI, BP, and SP. We'll become acquainted with them in Chapter 3. In addition, the EU contains temporary registers for holding operands for the ALU, and the FLAGS register whose individual bits reflect the result of a computation.

Bus Interface Unit (BIU)

The bus interface unit (BIU) facilitates communication between the EU and the memory or I/O circuits. It is responsible for transmitting addresses, data, and control signals on the buses. Its registers are named CS, DS, ES, SS, and IP; they hold addresses of memory locations. The **IP (instruction pointer)** contains the address of the next instruction to be executed by the EU.

The EU and the BIU are connected by an internal bus, and they work together. While the EU is executing an instruction, the BIU fetches up to six bytes of the next instruction and places them in the instruction queue. This operation is called *instruction prefetch*. The purpose is to speed up the processor. If the EU needs to communicate with memory or the peripherals, the BIU suspends instruction prefetch and performs the needed operations.

1.1.3

I/O Ports

I/O devices are connected to the computer through I/O circuits. Each of these circuits contains several registers called I/O ports. Some are used for data while others are used for control commands. Like memory locations, the I/O ports have addresses and are connected to the bus system. However, these addresses are known as I/O addresses and can only be used in input or output instructions. This allows the CPU to distinguish between an I/O port and a memory location.

I/O ports function as transfer points between the CPU and I/O devices. Data to be input from an I/O device are sent to a port where they can be read by the CPU. On output, the CPU writes data to an I/O port. The I/O circuit then transmits the data to the I/O device.

Serial and Parallel Ports

The data transfer between an I/O port and an I/O device can be 1 bit at a time (serial), or 8 or 16 bits at a time (parallel). A parallel port requires more wiring connections, while a serial port tends to be slower. Slow devices, like the keyboard, always connect to a serial port, and fast devices, like the disk drive, always connect to a parallel port. But some devices, like the printer, can connect to either a serial or a parallel port.

1.2

Instruction Execution

To understand how the CPU operates, let's look at how an instruction is executed. First of all, a machine instruction has two parts: an **opcode** and **operands**. The opcode specifies the type of operation, and the operands are often given as memory addresses to the data to be operated on. The CPU goes through the following steps to execute a machine instruction (the **fetch-execute cycle**):

Fetch

1. Fetch an instruction from memory.
2. Decode the instruction to determine the operation.
3. Fetch data from memory if necessary.

Execute

4. Perform the operation on the data.
5. Store the result in memory if needed.

To see what this entails, let's trace through the execution of a typical machine language instruction for the 8086. Suppose we look at the instruction that adds the contents of register AX to the contents of the memory word at address 0. The CPU actually adds the two numbers in the ALU and then stores the result back to memory word 0. The machine code is

```
00000001 00000110 00000000 00000000
```

Before execution, we assume that the first byte of the instruction is stored at the location indicated by the IP.

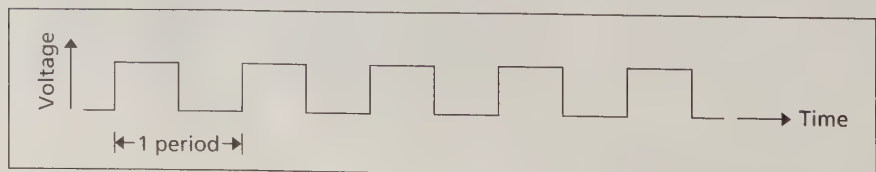
1. Fetch the instruction. To start the cycle, the BIU places a memory read request on the control bus and the address of the instruction on the address bus. Memory responds by sending the contents of the location specified—namely, the instruction code just given—over the data bus. Because the instruction code is four bytes and the 8086 can only read a word at a time, this involves two read operations. The CPU accepts the data and adds four to the IP so that the IP will contain the address of the next instruction.
2. Decode the instruction. On receiving the instruction, a decoder circuit in the EU decodes the instruction and determines that it is an ADD operation involving the word at address 0.
3. Fetch data from memory. The EU informs the BIU to get the contents of memory word 0. The BIU sends address 0 over the address bus and a memory read request is again sent over the control bus. The contents of memory word 0 are sent back over the data bus to the EU and are placed in a holding register.
4. Perform the operation. The contents of the holding register and the AX register are sent to the ALU circuit, which performs the required addition and holds the sum.
5. Store the result. The EU directs the BIU to store the sum at address 0. To do so, the BIU sends out a memory write request over the control bus, the address 0 over the address bus, and the sum to be stored over the data bus. The previous contents of memory word 0 are overwritten by the sum.

The cycle is now repeated for the instruction whose address is contained in the IP.

Timing

The preceding example shows that even though machine instructions are very simple, their execution is actually quite complex. To ensure that the steps are carried out in an orderly fashion, a clock circuit controls the processor

Figure 1.7 Train of Clock Pulses



by generating a train of **clock pulses** as shown in Figure 1.7. The time interval between two pulses is known as a **clock period**, and the number of pulses per second is called the **clock rate** or **clock speed**, measured in **megahertz (MHz)**. One megahertz is 1 million cycles (pulses) per second. The original IBM PC had a clock rate of 4.77 MHz, but the latest PS/2 model has a clock rate of 33 MHz.

The computer circuits are activated by the clock pulses; that is, the circuits perform an operation only when a clock pulse is present. Each step in the instruction fetch and execution cycle requires one or more clock periods. For example, the 8086 takes four clock periods to do a memory read and a multiplication operation may take more than seventy clock periods. If we speed up the clock circuit, a processor can be made to operate faster. However, each processor has a rated maximum clock speed beyond which it may not function properly.

1.3 I/O Devices

I/O devices are needed to get information into and out of the computer. The primary I/O devices are magnetic disks, the keyboard, the display monitor, and the printer.

Magnetic Disks

We've seen that the contents of RAM are lost when the computer is turned off, so magnetic disks are used for permanent storage of programs and data. There are two kinds of disks: **floppy disks** (also called diskettes) and **hard disks**. The device that reads and writes data on a disk is called a **disk drive**.

Floppy disks come in 5¼-inch or 3½-inch diameter sizes. They are lightweight and portable; it is easy to put a diskette away for safekeeping or use it on different computers. The amount of data a floppy disk can hold depends on the type; it ranges from 360 kilobytes to 1.44 megabytes. A **kilobyte (KB)** is 2^{10} bytes.

A hard disk and its disk drive are enclosed in a hermetically sealed container that is not removable from the computer; thus, it is also called a **fixed disk**. It can hold a lot more data than a floppy disk—typically 20, 40, to over 100 megabytes. A program can also access information on a hard disk much faster than a floppy disk.

Disk operations are covered in Chapter 19.

Keyboard

The keyboard allows the user to enter information into the computer. It has the keys usually found on a typewriter, plus a number of control and function keys. It has its own microprocessor that sends a coded signal to the computer whenever a key is pressed or released.

When a key is pressed, the corresponding key character normally appears on the screen. But interestingly enough, there is no direct connection between the keyboard and the screen. The data from the keyboard are received by the current running program. The program must send the data to the screen before a character is displayed. In Chapter 12 you will learn how to control the keyboard.

Display Monitor

The display monitor is the standard output device of the computer. The information displayed on the screen is generated by a circuit in the computer called a **video adapter**. Most adapters can generate both text characters and graphics images. Some monitors are capable of displaying in color.

We discuss text mode operations in Chapter 12, and cover graphics mode in Chapter 16.

Printers

Although monitors give fast visual feedback, the information is not permanent. Printers, however, are slow but provide more permanent output. Printer outputs are known as **hardcopies**.

The three common kinds of printers are *daisy wheel*, *dot matrix*, and *laser* printers. The output of a daisy wheel printer is similar to that of a typewriter. A dot matrix printer prints characters composed of dots; depending on the number of dots used per character, some dot matrix printers can generate near-letter-quality printing. The advantage of dot matrix printers is that they can print characters with different fonts as well as graphics.

The laser printer also prints characters composed of dots; however, the resolution is so high (300 dots per inch) that it has typewriter quality. The laser printer is expensive, but in the field of desktop publishing it is indispensable. It is also quiet compared to the other printers.

1.4 Programming Languages

The operations of the computer's hardware are controlled by its software. When the computer is on, it is always in the process of executing instructions. To fully understand the computer's operations, we must also study its instructions.

Machine Language

A CPU can only execute machine language instructions. As we've seen, they are bit strings. The following is a short machine language program for the IBM PC:

<i>Machine instruction</i>	<i>Operation</i>
10100001 00000000 00000000	Fetch the contents of memory word 0 and put it in register AX.
00000101 00000100 00000000	Add 4 to AX.
10100011 00000000 00000000	Store the contents of AX in memory word 0.

As you can well imagine, writing programs in machine language is tedious and subject to error!

Assembly Language

A more convenient language to use is **assembly language**. In assembly language, we use symbolic names to represent operations, registers, and memory locations. If location 0 is symbolized by A, the preceding program expressed in IBM PC assembly language would look like this:

<i>Assembly language instruction</i>	<i>Comment</i>
MOV AX,A	;fetch the contents of ;location A and ;put it in register AX
ADD AX,4	;add 4 to AX
MOV A,AX	;move the contents of AX ;into location A

A program written in assembly language must be converted to machine language before the CPU can execute it. A program called the **assembler** translates each assembly language statement into a single machine language instruction.

High-Level Languages

Even though it's easier to write programs in assembly language than machine language, it's still difficult because the instruction set is so primitive. That is why high-level languages such as FORTRAN, Pascal, C, and others were developed. Different high-level languages are designed for different applications, but they generally allow programmers to write programs that look more like natural language text than is possible in assembly language.

A program called a **compiler** is needed to translate a high-level language program into machine code. Compilation is more involved than assembling because it entails the translation of complex mathematical expressions and natural language commands into simple machine operations. A high-level language statement typically translates into many machine language instructions.

Advantages of High-Level Languages

There are many reasons why a programmer might choose to write a program in a high-level language rather than in assembly language.

First, because high-level languages are closer to natural languages, it's easier to convert a natural language algorithm to a high-level language program than to an assembly language program. For the same reason, it's easier to read and understand a high-level language program than an assembly language program.

Second, an assembly language program generally contains more statements than an equivalent high-level language program, so more time is needed to code the assembly language program.

Third, because each computer has its own unique assembly language, assembly language programs are limited to one machine, but a high-level language program can be executed on any machine that has a compiler for that language.

Advantages of Assembly Languages

The main reason for writing assembly language programs is efficiency: because assembly language is so close to machine language, a well-written assembly language program produces a faster, shorter machine language program. Also, some operations, such as reading or writing to specific memory locations and I/O ports, can be done easily in assembly language but may be impossible at a higher level.

Actually, it is not always necessary for a programmer to choose between assembly language and high-level languages, because many high-level languages accept subprograms written in assembly language. This means that

crucial parts of a program can be written in assembly language, with the rest written in a high-level language.

In addition to these considerations, there is another reason for learning assembly language. Only by studying assembly language is it possible to gain a feeling for the way the computer “thinks” and why certain things happen the way they do inside the computer. High-level languages tend to obscure the details of the compiled machine language program that the computer actually executes. Sometimes a slight change in a program produces a major increase in the run time of that program, or arithmetic overflow unexpectedly occurs. Such things can be understood on the assembly language level.

Even though here you will study assembly language specifically for the IBM PC, the techniques you will learn are typical of those used in any assembly language. Learning other assembly languages should be relatively easy after you have read this book.

1.5

An Assembly Language Program

To give an idea of what an assembly language program looks like, here is a simple example. The following program adds the contents of two memory locations, symbolized by A and B. The sum is stored in location SUM.

Program Listing PGM1_1.ASM

```
TITLE PGM1_1: SAMPLE PROGRAM
.MODEL SMALL
.STACK 100H
.DATA
A DW 2
B DW 5
SUM DW ?
.CODE
MAIN PROC
;initialize DS
MOV AX,@DATA
MOV DS,AX
;add the numbers
MOV AX,A           ;AX has A
ADD AX,B           ;AX has A+B
MOV SUM,AX         ;SUM = A+B
;exit to DOS
MOV AX,4C00H
INT 21H
MAIN ENDP
END MAIN
```

Assembly language programs consist of statements. A statement is either an instruction to be executed when the program is run, or a directive for the assembler. For example, `.MODEL SMALL` is an assembler directive that specifies the size of the program. `MOV AX,A` is an instruction. Anything that follows a semicolon is a comment, and is ignored by the assembler.

The preceding program consists of three parts, or segments: the *stack segment*, the *data segment*, and the *code segment*. They begin with the directives `.STACK`, `.DATA`, and `.CODE`, respectively.

The stack segment is used for temporary storage of addresses and data. If no stack segment is declared, an error message is generated, so there must be a stack segment even if the program doesn't utilize a stack.

Variables are declared in the data segment. Each variable is assigned space in memory and may be initialized. For example, `A DW 2` sets aside a memory word for a variable called A and initializes it to 2 (DW stands for "Define Word"). Similarly, `B DW 5` sets aside a word for variable B and initializes it to 5 (these initial values were chosen arbitrarily). `SUM DW ?` sets aside an uninitialized word for SUM.

A program's instructions are placed in the code segment. Instructions are usually organized into units called *procedures*. The preceding program has only one procedure, called MAIN, which begins with the line `MAIN PROC` and ends with line `MAIN ENDP`.

The main procedure begins and ends with instructions that are needed to initialize the DS register and to return to the DOS operating system. Their purpose is explained in Chapter 4. The instructions for adding A and B and putting the answer in SUM are as follows:

```
MOV AX,A           ;AX has A
ADD AX,B           ;AX has A+B
MOV SUM,AX         ;SUM = A+B
```

`MOV AX,A` copies the contents of word A into register AX. `ADD AX,B` adds the contents of B to it, so that AX now holds the total, 7. `MOV SUM,AX` stores the answer in variable SUM.

Before this program could be run on the computer, it would have to be assembled into a machine language program. The steps are explained in Chapter 4. Because there were no output instructions, we could not see the answer on the screen, but we could trace the program's execution in a debugger such as the DEBUG program.

Glossary

add-in board or card	Circuit board that connects to the motherboard, usually contains I/O circuits or additional memory
address	A number that identifies a memory location
address bus	The set of electrical pathways for address signals
arithmetic and logic unit, ALU	CPU circuit where arithmetic and logic operations are done
assembler	A program that translates an assembly language program into machine language
assembly language	Symbolic representation of machine language
binary digit	A symbol that can have value 0 or 1
bit	Binary digit
bus	A set of wires or connections connecting the CPU, memory, and I/O ports

bus interface unit, BIU	Part of the CPU that facilitates communication between the CPU, memory, and I/O ports
byte	8 bits
central processing unit, CPU	The main processor circuit of a computer
clock period	The time interval between two clock pulses
clock pulse	An electrical signal that rises from a low voltage to a high voltage and down again to a low voltage, used to synchronize computer circuit operations
clock rate	The number of clock pulses per second, measured in megahertz (MHz)
clock speed	Clock rate
compiler	A program that translates a high-level language to machine language
contents	The data stored in a register or memory location
control bus	The set of electrical paths for control signals
data bus	The set of electrical paths for data signals
digital circuits	Circuits that operate on discrete voltage levels
disk drive	The device that reads and writes data on a disk
execution unit, EU	Part of the CPU that executes instructions
expansion slots	Connectors in the motherboard where other circuit boards can be attached
fetch-execute cycle	Cycle the CPU goes through to execute an instruction
firmware	Software supplied by the computer manufacturer, usually stored in ROM
fixed disk	Nonremovable disk, made of metal
floppy disk	Removable, flexible disk
hardcopy	Printer output
hard disk	Fixed disk
I/O devices	Devices that handle input and output data of the computer; typical I/O devices are display monitor, disk drive, and printer
I/O ports	Circuits that function as transfer points between the CPU and I/O devices
instruction pointer, IP	A CPU register that contains the address of the next instruction
instruction set	The instructions the CPU is capable of performing
kilobyte, KB	2^{10} or 1024 bytes
machine language	Instructions coded as bit strings: the language of the computer

mega	A unit that usually denotes 1 million, but in computer terminology 1 mega is 2^{20} (or 1,048,576)
megabyte, MB	2^{20} or 1,048,576 bytes
megahertz, MHz	1,000,000 cycles per second
memory byte (circuit)	A memory circuit that can store one byte
memory location	A memory byte or memory word
memory word	Two memory bytes
microprocessor	A processing unit fabricated on a single circuit chip
motherboard	The main circuit board of the computer
opcode	Numeric or symbolic code denoting the type of operation for an instruction
operand	The data specified in an instruction
peripheral (device)	I/O device
random access memory, RAM	Memory circuits that can be read or written
read-only memory, ROM	Memory circuits that can only be read
register	A CPU circuit for storing information
system board	Motherboard
video adapter	Computer circuit that converts computer data into video signals for the display monitor
word	16 bits

Exercises

1. Suppose memory bytes 0–4 have the following contents:

Address	Contents
0	01101010
1	11011101
2	00010001
3	11111111
4	01010101

- a. Assuming that a word is 2 bytes, what are the contents of
 - the memory word at address 2?
 - the memory word at address 3?
 - the memory word whose high byte is the byte at address 2?
- b. What is
 - bit 7 of byte 2?
 - bit 0 of word 3?
 - bit 4 of byte 2?
 - bit 11 of word 2?

2. A **nibble** is four bits. Each byte is composed of a high nibble and a low nibble, similar to the high and low bytes of a word. Using the data in exercise 1, give the contents of
 - a. the low nibble of byte 1.
 - b. the high nibble of byte 4.
3. The two kinds of memory are RAM and ROM. Which kind of memory
 - a. holds a user's program?
 - b. holds the program used to start the machine?
 - c. can be changed by the user?
 - d. retains its contents, even when the power is turned off?
4. What is the function of
 - a. the microprocessor?
 - b. the buses?
5. The two parts of the microprocessor are the EU and the BIU.
 - a. What is the function of the EU?
 - b. What is the function of the BIU?
6. In the microprocessor, what is the function of
 - a. the IP?
 - b. the ALU?
7.
 - a. What are the I/O ports used for?
 - b. How are they different from memory locations?
8. What is the maximum length (in bytes) of an instruction for the 8086-based IBM PC?
9. Consider a machine language instruction that moves a copy of the contents of register AX in the CPU to a memory word. What happens during
 - a. the fetch cycle?
 - b. the execute cycle?
10. Give
 - a. three advantages of high-level language programming.
 - b. the primary advantage of assembly language programming.